# استرجاع وتخزين المعلومات (Information Storage and Retrieval - IR)

الملخص الدراسي الشامل والموسوعة المرجعية التنفيذية لامتحان الميدتيرم

المادة: استرجاع المعلومات | النطاق: الفصول 1، 2، 3، 5 | الفهرس المعكوس | مؤشرات القفز | التجذير والتوحيد

## 01 مقدمة في استرجاع المعلومات والفهرس المعكوس (الفصل 1)

**التعريف الأكاديمي المعتمد لاسترجاع المعلومات (IR)**

هو عملية العثور على مواد (غالباً وثائق) ذات طبيعة غير مهيكلة (غالباً نصوص مكتوبة) تلبي حاجة معلوماتية لدى المستخدم من بين مجموعات تخزينية ضخمة مخزنة على الحواسيب.

### 1.1 الفرق بين الحاجة المعلوماتية والاستعلام (Need vs. Query)

- **الحاجة المعلوماتية (Information Need):** الموضوع أو الرغبة المعرفية الكامنة في عقل المستخدم (مثال: "أريد معرفة هل شرب القهوة يرفع ضغط الدم").
- **الاستعلام (Query):** الكلمات الفعلية التي يكتبها المستخدم في مربع البحث للتعبير عن تلك الحاجة (مثال: `coffee blood pressure`). وغالباً ما تكون غير دقيقة أو مبهمة.

### 1.2 استرجاع المعلومات مقابل قواعد البيانات التقليدية

| وجه المقارنة | قواعد البيانات العلائقية (RDBMS) | أنظمة استرجاع المعلومات (IR) |
|---|---|---|
| **طبيعة البيانات** | بيانات مهيكلة في جداول وحقول محددة وصارمة. | نصوص حرة غير مهيكلة أو شبه مهيكلة. |
| **المطابقة والدلالة** | دلالات واضحة ومطابقة منطقية صارمة (Exact Match). | لغة طبيعية غامضة ومطابقة تقريبية تعتمد على الترتيب والصلة (Relevance Ranking). |
| **لغة الاستعلام** | لغة تصريحية دقيقة (SQL). | كلمات مفتاحية، معاملات بولیانية، أو نصوص حرة. |
| **النتائج المسترجعة** | مجموعة غير مرتبة تطابق شرط WHERE تماماً. | قائمة وثائق مرتبة تنازلياً حسب درجة الأهمية والملاءمة للبحث. |

### 1.3 مصفوفة وقوع الكلمات (Term-Document Matrix) ومأزق التناثر

- **المصفوفة الثنائية:** جدول يمثل صفوفه الكلمات ($V$) وأعمدته الوثائق ($N$)؛ وتوضع القيمة 1 إذا وردت الكلمة في الوثيقة و 0 إذا لم ترد.

- **مأزق التناثر (Sparsity Problem):** إذا كان لدينا 1,000,000 وثيقة و 500,000 كلمة، فإن حجم المصفوفة هو 500 مليار خانة! أكثر من 99.9% منها أصفار. تخزين هذه المصفوفة كثيفة البيانات في الذاكرة مستحيل تماماً وهدر هائل للموارد.

### 1.4 الفهرس المعكوس (Inverted Index): الهيكل المعياري

يقوم الفهرس المعكوس بحفظ الخانات غير الصفرية فقط، بحيث يربط كل كلمة بقائمة أرقام الوثائق التي ظهرت فيها.

- **القاموس (Dictionary / Vocabulary):** قائمة تحوي كافة الكلمات الفريدة مرتبة أبجدياً مع تردد الوثائق ($df$). يوضع غالباً في الذاكرة العشوائية (RAM) باستخدام B-Tree أو Hash Table.
- **قوائم النشر (Postings Lists):** لكل كلمة في القاموس، قائمة متغيرة الطول بأرقام الوثائق (DocIDs) التي وردت فيها تلك الكلمة، وتكون مرتبة **تصاعدياً دائماً**.

### 1.5 مراحل بناء الفهرس المعكوس

1. **تحليل الوثائق (Parsing):** استخراج سلاسل الحروف وتحديد الترميز (UTF-8) من الملفات المختلفة (PDF, HTML, Word).
2. **التقطيع اللغوي (Tokenization):** تقسيم تدفق الحروف إلى وحدات نصية وكلمات منفصلة (Tokens).
3. **المعالجة اللغوية (Linguistic Processing):** توحيد حالة الحروف، معالجة كلمات التوقف، والتجذير لتحويل الكلمات إلى مفردات معيارية (Terms).
4. **الفهرسة والترتيب:** استخراج أزواج (الكلمة، رقم الوثيقة)، ترتيبها أبجدياً حسب الكلمة ثم تصاعدياً حسب رقم الوثيقة، وتجميعها في قوائم النشر.

## 02 معالجة الاستعلامات البولينية، التحسين واستعلامات العبارات (المحاضرة 2)

### 2.1 خوارزمية دمج وتقاطع قوائم النشر (Postings Merge)

لتقييم استعلام بولياني يحتوي على AND (مثل `Brutus AND Caesar`)، يتم المرور المتزامن على قائمتي النشر المرتبتين باستخدام مؤشرين ($p_1, p_2$).

```
Algorithm INTERSECT(p1, p2): answer = [] while p1 != NIL and p2 != NIL: if DocID(p1) == DocID(p2): answer.append(DocID(p1)) p1 = p1.next p2 = p2.next else if DocID(p1) < DocID(p2): p1 = p1.next else: p2 = p2.next return answer
```

> **التعقيد الزمني (Time Complexity)**
> نظراً لأن القائمتين مرتبتان تصاعدياً مسبقاً، فإن كل مؤشر يمر على القائمة مرة واحدة خطياً. التعقيد الزمني هو **O(x + y)** خطي تماماً، حيث x و y هما طولا القائمتين.

### 2.2 تحسين ترتيب تنفيذ الاستعلامات البولينية

- **قاعدة الترتيب الأمثل:** في استعلامات AND المتعددة، يتم ترتيب الكلمات دائماً وفقاً لـ **التردد التصاعدي للوثائق (Increasing df)**؛ أي البدء بأقصر قوائم النشر طولاً.
- **مثال امتحاني شهير:**
  - `Calpurnia` ($df = 4$)
  - `Brutus` ($df = 20,000$)
  - `Caesar` ($df = 40,000$)
  - الترتيب الأمثل للتنفيذ هو: `(Calpurnia AND Brutus) AND Caesar`. لأن التقاطع الأول ينتج قائمة لا تزيد عن **4 وثائق فقط**! وتقاطع 4 وثائق مع قائمة Caesar يستغرق زمناً يقارب الصفر.

### 2.3 استعلامات العبارات المركبة: فهرس الزوجين مقابل الفهرس الموضعي

| نوع الفهرس | طريقة العمل | المزايا والعيوب |
|---|---|---|
| **فهرس الكلمتين (Biword Index)** | فهرسة كل كلمتين متجاورتين كمفردة مستقلة في القاموس (مثل "friends romans"، "romans countrymen"). | **المزايا:** سرعة فائقة للعبارات المكونة من كلمتين. **العيوب:** انفجار هائل في حجم القاموس ($V^2$)، احتمالية ظهور نتائج خاطئة للعبارات الطويلة، وعجزه عن استعلامات التقارب. |
| **الفهرس الموضعي (Positional Index)** | حفظ أرقام مواقع الكلمات بدقة داخل كل وثيقة: `term -> docID: `. | **المزايا:** يحل استعلامات العبارات مهما كان طولها واستعلامات التقارب (`/k`). وهو الحل المعياري المتبع عالمياً. **العيوب:** تضخم قوائم النشر من 2 إلى 4 أضعاف (35-50% من حجم النص الأصلي). |
| **المخطط الهجين (Scheme Combination)** | فهرسة العبارات شديدة التكرار كزوجين (مثل "New York")، واستخدام الفهرس الموضعي للعبارات النادرة. | أفضل توازن ممكن بين سرعة التنفيذ وسعة التخزين. |

### 2.4 استعلامات التقارب (`Proximity Queries: `/k)

استعلام مثل `employment /3 federal` يبحث عن الوثائق التي تظهر فيها الكلمتان بفاصل لا يتجاوز 3 كلمات بأي ترتيب ($|pos_1 - pos_2| \le 3$). يقوم الفهرس الموضعي بفحص تطابق رقم الوثيقة ثم مقارنة الفارق بين المواقع.

## 03 تسريع دمج القوائم ومؤشرات القفز (المحاضرة 3)

### 3.1 آلية مؤشرات القفز (Skip Pointers) وأهميتها

عند تقاطع قائمتين طويلتين، يؤدي الفحص المتسلسل خطوة بخطوة إلى هدر فحص آلاف الوثائق التي لن تتطابق. تقوم **مؤشرات القفز** بإنشاء مسارات اختصار للأمام لتخطي أجزاء كبيرة من القائمة دفعة واحدة.

> **قاعدة اتخاذ قرار القفز**
>
> أثناء مقارنة المؤشرين $$p_1$$ و $$p_2$$: إذا كان لدى $$p_1$$ مؤشر قفز وكان **skip(p1) ≤ DocID(p2)**، يتم القفز مباشرة إلى `skip(p1)`! أما إذا كان `skip(p1) > DocID(p2)`، فلا نقفز (لتجنب تجاوز وثيقة مطابقة محتملة) ونتقدم خطوة واحدة عادية.

### 3.2 موضع وضع مؤشرات القفز: القاعدة الذهبية

- **المعادلة والمفاضلة:** زيادة مؤشرات القفز تعطي فرصاً أكبر للقفز ولكنها تزيد مساحة التخزين وتكلفة الفحص؛ وقلة المؤشرات توفر التخزين ولكنها تفوت فرص التخطي السريع.
- **القاعدة القياسية (Heuristic):** في قائمة نشر طولها $P$، يتم وضع مؤشرات قفز متباعدة بمسافات متساوية كل **√P** عنصر!
- **مثال عملي:** إذا كان طول القائمة $$P = 10,000$$ وثيقة، نضع مؤشر قفز كل $\sqrt(10, 0) = 100$ عنصر، مما ينتج حوالي 100 مؤشر قفز.
- **محدودية الفهارس الديناميكية:** تعمل مؤشرات القفز بكفاءة مثالية على الفهارس الثابتة (Static)؛ أما في الفهارس المتغيرة بكثرة، فإن عمليات الإضافة والحذف تفسد المسافات وتتطلب إعادة حساب مكلفة.

## 04 مفردات القاموس والتوحيد المعياري للنصوص (المحاضرة 5)

### 4.1 تحديات التقطيع النصي (Tokenization)

- **ترميز المحارف (Character Encodings):** التعرف الصحيح على ترميز الملفات (UTF-8, ASCII) لتجنب ظهور رموز غير مفهومة (Mojibake).
- **الشرطات الواصلة (Hyphens):** الكلمات المركبة (مثل `state-of-the-art`, `Hewlett-Packard`) تفرض تحدياً: هل تُفهرس ككلمة واحدة أم تُفصل إلى كلمات مستقلة؟
- **الفاصلة العليا (Apostrophes):** الاختصارات (`aren't`) وحالات الملكية (`children's`) والأسماء الأيرلندية (`O'Neill`).
- **الأرقام والصيغ:** أرقام الطرازات (`Boeing 747`)، التواريخ، والصيغ الكيميائية تمتلك قدرة تمييزية فائقة ويجب الحفاظ عليها في الفهرس.
- **خصوصية اللغات:** الصينية واليابانية لا تحتويان على مسافات بين الكلمات وتتطلبان تقطيعاً صرفياً أو N-grams؛ بينما تلتحق بلغة الضاد (العربية) سوابق لواصق مثل الواو وأداة التعريف "ال" في بداية الأسماء.

### 4.2 كلمات التوقف (Stop Words): التقليد القديم مقابل التوجه الحديث

- **كلمات التوقف:** كلمات وظيفية شديدة التكرار وقليلة الفائدة الدلالية في التمييز بين الوثائق (مثل: `the`, `a`, `in`, `of`, `to`, `من`, `في`, `على`).
- **المنهج التقليدي:** حذف قائمة من 30 إلى 150 كلمة توقف لتوفير مساحة التخزين على القرص.
- **التوجه الحديث في محركات البحث الكبرى:** الإبقاء على كافة كلمات التوقف في الفهرس، وذلك لسببين: (1) تقنيات الضغط الحديثة حلت مشكلة المساحة؛ (2) يحتاج المستخدمون للبحث عن عبارات دقيقة تتكون معظمها من كلمات توقف (مثل `"To be or not to be"`, `"Let it be"`).

### 4.3 توحيد النصوص: حالة الأحرف ومجموعات التكافؤ

- **مجموعات التكافؤ (Equivalence Classes):** دمج الأشكال المتعددة للكلمة الواحدة في رمز موحد (مثل تحويل `U.S.A.` و `USA` إلى مصطلح واحد).
- **توحيد حالة الأحرف (Case Folding):** تحويل الحروف الإنجليزية جميعها إلى أحرف صغيرة (Lowercase). محاذيره: قد يخلط بين أسماء الأعلام والكلمات العادية (مثل `US` كدولة و `us` كضمير؛ و `Bush` كرئيس و `bush` كشجيرة).
- **خوارزمية ساوندكس (Soundex):** خوارزمية صوتية تحول الكلمات إلى رموز وفقاً لنطقها الصوتي باللغة الإنجليزية لمطابقة الأسماء المتشابهة صوتياً رغم اختلاف كتابتها (مثل `Smith` و `Smyth`).

### 4.4 التجذير الصرفي مقابل التحليل المعجمي (Stemming vs. Lemmatization)

| وجه المقارنة | التجذير الصرفي الخشن (Stemming) | التحليل المعجمي الدقيق (Lemmatization) |
|---|---|---|
| **المنهجية** | قص تجريبي وسريع لنهايات الكلمات وزوائدها دون قواعد نحوية أو قواميس. | تحليل صرفي ونحوي كامل بالاعتماد على معجم وقواعد وسياق موقع الكلمة (POS). |
| **المخرج الناتج** | غالباً ما ينتج جذراً غير حقيقي ولا يمثل كلمة في القاموس (مثل: `automate, automatic, automation` → `automat`). | ينتج دائماً الكلمة المعجمية الأصلية الصحيحة نحوياً وتسمى **Lemma**. |

| | | |
|---|---|---|
| **حساسية السياق** | أعمى للسياق النحوي ويعامل كل كلمة بنفس القاعدة الجامدة. | يدرك السياق؛ فيحول `saw` إلى `see` إذا وردت كفعل، ويتركها `saw` (منشار) إذا كانت اسماً. ويحول `better` إلى `good`. |
| **السرعة والتعقيد** | فائق السرعة وخفيف جداً على الذاكرة ومناسب للأنظمة الضخمة. | معقد حوسبياً ويستهلك معالجة وذاكرة أكبر لقراءة القواميس النحوية. |

### 4.5 خوارزمية بورتر (Porter Stemmer) وأخطاء التجذير

- **خوارزمية بورتر (1980):** الخوارزمية القياسية الأكثر انتشاراً لتجذير الإنجليزية، وتتألف من 5 مراحل متتالية من قواعد قص اللواحق استناداً لمقياس المقاطع الصوتية $m$.
- **خطأ الإفراط في التجذير (Over-stemming):** قص أجزاء زائدة مما يؤدي لدمج كلمات مختلفة المعنى في جذر واحد خطأً (مثل دمج `universe` و `university` إلى `univers`؛ أو `policy` و `police` إلى `polic`)، مما يقلل من دقة البحث (Precision).
- **خطأ التقصير في التجذير (Under-stemming):** الفشل في إرجاع الكلمات المرتبطة لنفس الجذر المشترك (مثل عدم دمج `adhere` و `adhesion`)، مما يضر بمعدل الاستدعاء (Recall).
- **قانون هيبس (Heaps' Law):** حجم المفردات $V$ ينمو باستمرار مع نمو حجم الوثائق $N$ وفق قانون القوة $V = k$ $N^\beta$ (حيث $\beta \approx 0.4 - 0.6$)، مما يؤكد أن مفردات القاموس لا تتوقف أبداً عن التوسع.